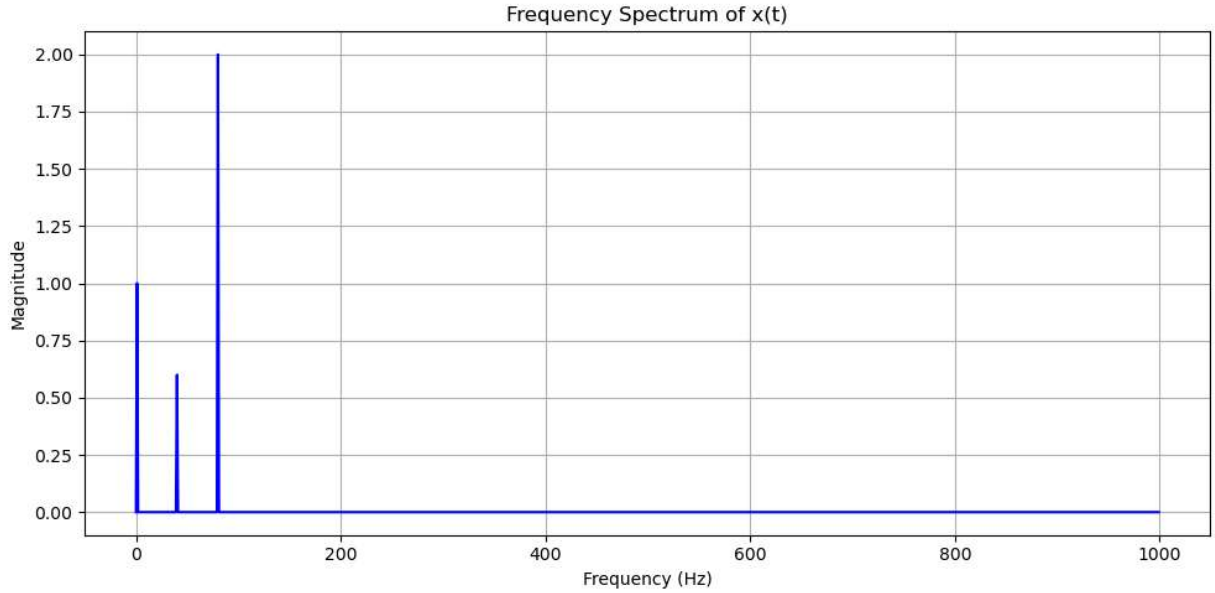
In [ ]:
```
# 1 (b)
```

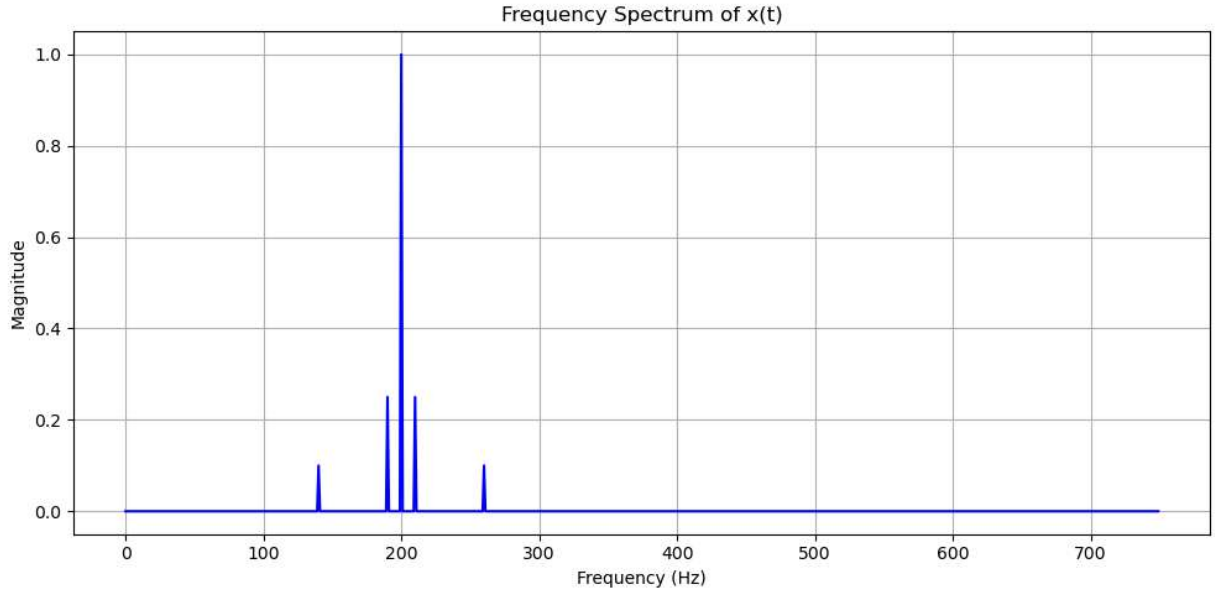
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
Fs = 2000            # Sampling frequency (Hz)
T = 1.0              # Signal duration (seconds)
N = int(Fs * T)      # Total samples (2000)

# Time vector
t = np.linspace(0, T, N, endpoint=False)

# Signal definition
x = (
    np.sin(2 * np.pi * 1 * t)
    + 2 * np.sin(2 * np.pi * 80 * t)
    + 0.6 * np.sin(2 * np.pi * 40 * t)
)

# FFT computation
X = np.fft.fft(x)
freqs = np.fft.fftfreq(N, 1/Fs)

# positive frequencies
half_N = N // 2
X_mag = np.abs(X[:half_N]) * 2 / N
freqs_pos = freqs[:half_N]

# Plotting
plt.figure(figsize=(10,5))
plt.plot(freqs_pos, X_mag, color='b')
plt.title("Frequency Spectrum of x(t)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [2]:
```
# 2 (d)
```

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
Fs = 1500            # Sampling frequency (Hz)
T = 1.0              # Signal duration (seconds)
N = int(Fs * T)      # Total samples (2000)

# Time vector
t = np.linspace(0, T, N, endpoint=False)

# Signal definition
x = (( 1+
    0.5*np.sin(2 * np.pi * 10 * t)
    + 0.2 * np.sin(2 * np.pi * 60 * t)
)*np.cos(2 * np.pi * 200 * t))

# FFT computation
X = np.fft.fft(x)
freqs = np.fft.fftfreq(N, 1/Fs)

# positive frequencies
half_N = N // 2
X_mag = np.abs(X[:half_N]) * 2 / N
freqs_pos = freqs[:half_N]

# Plotting
plt.figure(figsize=(10,5))
plt.plot(freqs_pos, X_mag, color='b')
plt.title("Frequency Spectrum of x(t)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```



In [ ]: